

Cahier des Charges Technique et Fonctionnel

Projet : Plateforme DevSecOps ChatOps AI

1. Présentation du projet

1.1 Nom du projet

ChatOps AI — Plateforme DevSecOps d'automatisation et de supervision d'un chatbot IA

1.2 Contexte

Le projet consiste à concevoir, déployer et superviser une plateforme DevSecOps complète permettant d'héberger un chatbot basé sur l'intelligence artificielle.

L'objectif est de mettre en œuvre une architecture moderne orientée Cloud, automatisée et sécurisée, reproduisant les standards utilisés dans les environnements professionnels.

Le projet intègre :

- Un backend API développé avec FastAPI
 - Une interface frontend développée avec Next.js
 - Une infrastructure conteneurisée avec Docker
 - Une orchestration avec Docker Compose
 - Une automatisation CI/CD avec GitHub Actions
 - Un déploiement sur AWS EC2
 - Une supervision avec Prometheus et Grafana
 - Des pratiques DevSecOps
 - Une architecture extensible orientée IA
-

2. Objectifs du projet

2.1 Objectifs principaux

- Automatiser le cycle complet de déploiement d'une application IA
- Fournir une architecture DevOps moderne et industrialisée
- Déployer une application scalable et maintenable
- Centraliser les outils de monitoring
- Garantir une haute disponibilité des services
- Sécuriser les workflows CI/CD
- Simuler une infrastructure d'entreprise réelle

2.2 Objectifs techniques

- Conteneurisation complète de l'application
 - Automatisation des builds et déploiements
 - Mise en place d'un monitoring temps réel
 - Gestion des variables sensibles via GitHub Secrets
 - Déploiement Cloud automatisé
 - Séparation frontend/backend
 - Mise en place d'une architecture extensible
-

3. Périmètre fonctionnel

3.1 Fonctionnalités Frontend

Interface utilisateur

- Interface responsive moderne
- Interface de discussion IA
- Tableau de bord DevOps
- Affichage des métriques système
- Communication temps réel avec le backend
- Navigation fluide entre les modules

Technologies utilisées

- Next.js
 - React
 - TypeScript
 - Tailwind CSS
-

3.2 Fonctionnalités Backend

API REST

- Endpoint principal de chatbot
- Endpoints de monitoring
- Endpoint de healthcheck
- Gestion des requêtes asynchrones
- Architecture compatible OpenAI
- Gestion des erreurs
- Logs applicatifs

Technologies utilisées

- Python
- FastAPI
- Uvicorn

3.3 Fonctionnalités DevOps

CI/CD

- Build automatique des images Docker
- Tests automatiques
- Déploiement automatique sur AWS EC2
- Synchronisation via SSH
- Pipeline GitHub Actions

Infrastructure

- Docker Compose
- AWS EC2
- Docker Engine
- Reverse proxy potentiel avec Nginx

Monitoring

- Prometheus
 - Grafana
 - Collecte de métriques
 - Dashboards de supervision
-

4. Architecture technique

4.1 Architecture globale

```
Utilisateur
|
▼
Frontend Next.js
|
▼
API FastAPI
|
▼
Services IA / API externes
```

```
GitHub
|
GitHub Actions
|
Déploiement automatique
|
```

```
graph TD
    AWS[AWS EC2] --> Docker[Docker Compose]
    Docker --> Monitoring[Monitoring Prometheus/Grafana]
```

4.2 Architecture logicielle

Frontend

Élément	Description
Framework	Next.js
Langage	TypeScript
UI	React
Style	Tailwind CSS
Communication	API REST

Backend

Élément	Description
Framework	FastAPI
Langage	Python
Serveur ASGI	Uvicorn
API	REST
Monitoring	Endpoint /metrics

DevOps

Élément	Description
CI/CD	GitHub Actions
Conteneurisation	Docker
Orchestration	Docker Compose
Cloud	AWS EC2
Monitoring	Grafana + Prometheus

5. Arborescence du projet

```
.
├── backend/
│   ├── app/
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── app/
│   ├── public/
│   ├── package.json
│   └── tailwind.config.js
├── monitoring/
│   └── prometheus.yml
├── .github/
│   └── workflows/
│       ├── ci.yml
│       └── deploy.yml
├── docker-compose.yml
├── docker-compose.monitoring.yml
├── README.md
└── DEPLOYMENT.md
```

6. Description des composants

6.1 Frontend Next.js

Rôle

Le frontend fournit l'interface utilisateur permettant :

- d'interagir avec le chatbot IA,
- d'accéder aux dashboards,
- de visualiser les métriques,
- de consulter les états applicatifs.

Fonctionnalités attendues

- Page d'accueil
- Module Chat IA
- Dashboard de monitoring
- Gestion des erreurs API

- Responsive Design
- Optimisation SEO
- Build production optimisé

Contraintes techniques

- Compatibilité mobile et desktop
 - Temps de chargement inférieur à 3 secondes
 - Communication HTTPS
 - Architecture composants réutilisables
-

6.2 Backend FastAPI

Rôle

Le backend centralise la logique métier et les interactions avec les services IA.

Endpoints principaux

Endpoint	Méthode	Description
/	GET	Vérification API
/health	GET	Vérification santé service
/metrics	GET	Métriques Prometheus
/chat	POST	Requête chatbot

Fonctionnalités attendues

- Gestion des requêtes IA
- Journalisation des événements
- Gestion des erreurs
- Gestion des variables d'environnement
- Monitoring applicatif
- Gestion des performances

Contraintes techniques

- Temps de réponse inférieur à 500 ms
 - Architecture asynchrone
 - Scalabilité horizontale
 - Faible consommation mémoire
-

6.3 Docker et conteneurisation

Objectifs

- Isolation des services

- Reproductibilité des environnements
- Déploiement simplifié
- Standardisation infrastructure

Conteneurs prévus

Service	Description
frontend	Application Next.js
backend	API FastAPI
prometheus	Monitoring métriques
grafana	Dashboard supervision

Contraintes

- Images légères
 - Multi-stage build recommandé
 - Variables sensibles externalisées
 - Réseau Docker sécurisé
-

7. CI/CD — GitHub Actions

7.1 Objectifs

- Automatiser les tests
 - Automatiser les builds Docker
 - Déployer automatiquement en production
 - Réduire les erreurs humaines
-

7.2 Pipeline CI

Étapes

1. Checkout du code
2. Installation des dépendances
3. Exécution des tests
4. Build Docker
5. Validation pipeline

Technologies

- GitHub Actions
 - Pytest
 - Docker
-

7.3 Pipeline CD

Étapes

1. Connexion SSH à AWS EC2
2. Transfert des fichiers
3. Arrêt des conteneurs existants
4. Reconstruction Docker
5. Relancement des services

Variables sensibles

Secret GitHub	Description
HOST	IP du serveur
SSH_KEY	Clé SSH privée
OPENAI_API_KEY	Clé API OpenAI

8. Infrastructure Cloud AWS

8.1 Environnement cible

Cloud Provider

- AWS

Service principal

- EC2 Ubuntu Server

Configuration minimale recommandée

Ressource	Valeur minimale
CPU	2 vCPU
RAM	4 Go
Stockage	20 Go SSD
OS	Ubuntu 22.04

8.2 Services AWS utilisés

Service	Rôle
EC2	Hébergement applicatif

Service	Rôle
Security Groups	Sécurité réseau
Elastic IP	Adresse IP fixe
IAM	Gestion permissions

8.3 Ports réseau

Port	Usage
22	SSH
80	HTTP
443	HTTPS
3000	Grafana
8000	API FastAPI
9090	Prometheus

9. Monitoring et observabilité

9.1 Prometheus

Rôle

- Collecte des métriques système
- Scraping des endpoints
- Historisation des données

Métriques collectées

- CPU
- RAM
- Temps de réponse API
- Disponibilité services
- Utilisation conteneurs

9.2 Grafana

Rôle

- Visualisation des métriques
- Création de dashboards

- Supervision temps réel

Dashboards attendus

- Santé application
 - Consommation ressources
 - Monitoring Docker
 - Disponibilité API
 - Temps de réponse backend
-

10. Sécurité — DevSecOps

10.1 Sécurité applicative

Mesures attendues

- Variables sensibles dans .env
 - GitHub Secrets
 - Gestion HTTPS
 - Validation des entrées utilisateur
 - Protection API
 - Isolation conteneurs
-

10.2 Sécurité infrastructure

Mesures attendues

- Pare-feu AWS Security Groups
 - Clés SSH sécurisées
 - Restriction des ports
 - Mises à jour système régulières
 - Gestion des permissions Linux
-

10.3 Sécurité CI/CD

Bonnes pratiques

- Secrets GitHub chiffrés
 - Pipeline sécurisé
 - Validation avant déploiement
 - Vérification des images Docker
 - Scans de vulnérabilités recommandés
-

11. Base de données (évolution future)

11.1 Objectif futur

Ajouter une couche de persistance permettant :

- Historique conversations
 - Authentification utilisateurs
 - Logs applicatifs
 - Analytics IA
-

11.2 Technologies recommandées

Usage	Technologie
SQL	PostgreSQL
Cache	Redis
Logs	Elasticsearch

12. Exigences non fonctionnelles

12.1 Performance

Exigence	Valeur
Disponibilité	99 %
Temps réponse API	< 500 ms
Temps chargement frontend	< 3 sec
Déploiement CI/CD	< 10 min

12.2 Scalabilité

Le système devra pouvoir :

- Ajouter plusieurs instances backend
 - Supporter plusieurs utilisateurs simultanés
 - Être migré vers Kubernetes
 - Supporter un load balancer
-

12.3 Maintenabilité

Le projet devra :

- Être documenté
 - Être versionné avec Git
 - Respecter une architecture modulaire
 - Posséder des logs exploitables
 - Faciliter les mises à jour
-

13. Kubernetes — Évolution future recommandée

13.1 Objectifs

- Orchestration avancée
 - Auto-scaling
 - Haute disponibilité
 - Rolling updates
-

13.2 Composants recommandés

Composant	Rôle
Kubernetes	Orchestration
Helm	Packaging
Ingress Nginx	Reverse proxy
Cert-Manager	HTTPS automatique
ArgoCD	GitOps

14. Tests

14.1 Types de tests

Type	Description
Unitaires	Validation logique métier
Intégration	Vérification communication services
End-to-End	Validation workflow complet

Type	Description
Charge	Tests de performance
Sécurité	Détection vulnérabilités

14.2 Outils recommandés

Usage	Outil
Backend tests	Pytest
Frontend tests	Jest
API tests	Postman
Charge	Locust
Sécurité	Trivy / SonarQube

15. Documentation

15.1 Documentation technique

Le projet devra contenir :

- README détaillé
- Guide installation
- Guide déploiement
- Architecture diagrammes
- Variables d'environnement
- Documentation API

15.2 Documentation utilisateur

- Manuel d'utilisation
 - Guide dashboard
 - Guide monitoring
 - Guide maintenance
-

16. Livrables attendus

16.1 Livrables techniques

- Code source complet
 - Dockerfiles
 - docker-compose.yml
 - Pipelines GitHub Actions
 - Scripts déploiement
 - Configuration monitoring
-

16.2 Livrables documentation

- Cahier des charges
 - README technique
 - Documentation architecture
 - Guide utilisateur
 - Guide DevOps
-

17. Planning prévisionnel

Phase	Durée estimée
Analyse besoins	2 jours
Développement backend	4 jours
Développement frontend	4 jours
Dockerisation	2 jours
CI/CD GitHub Actions	2 jours
Déploiement AWS	2 jours
Monitoring	2 jours
Documentation	2 jours
Tests et validation	3 jours

18. Risques projet

Risque	Impact	Solution
Erreur CI/CD	Élevé	Validation pipelines

Risque	Impact	Solution
Mauvaise gestion secrets	Critique	GitHub Secrets
Surcharge serveur	Moyen	Monitoring + scaling
Vulnérabilités Docker	Élevé	Scan sécurité
Panne EC2	Moyen	Sauvegardes AMI

19. Améliorations futures recommandées

DevOps avancé

- Kubernetes
- Terraform
- Ansible
- GitOps avec ArgoCD
- Monitoring avancé
- Centralisation logs ELK

IA

- Intégration OpenAI complète
- Mémoire conversationnelle
- Fine-tuning modèles
- IA multi-agents

Sécurité

- OAuth2/JWT
- WAF AWS
- Reverse Proxy Nginx
- HTTPS automatique Let's Encrypt
- Scan sécurité automatisé

20. Conclusion

Le projet ChatOps AI constitue une plateforme DevSecOps complète permettant de démontrer :

- les compétences DevOps,
- l'automatisation CI/CD,
- le déploiement Cloud,
- la supervision d'infrastructure,
- la conteneurisation,
- et l'intégration d'applications IA modernes.

Cette architecture est conçue pour être évolutive, maintenable et proche des standards utilisés dans les environnements professionnels.

Le projet peut servir :

- de projet portfolio DevOps,
- de démonstration technique,
- de support pédagogique,
- ou de base pour une plateforme SaaS IA.